

Mindmate

Final Year Project



Supervisor: <Mr Muzammil Rehman>

Presented by

F22BINFT1M01150

Kashmala Nawaz



Department of Information Technology (DIT)

The Islamia University Of Bahawalpur

Topic of Project

Mindmate

By

Kashmala Nawaz

Project submitted to

Department of Information Technology,

The Islamia University of Bahawalpur.

in partial fulfillment of the requirements for the degree of

**BACHELOR OF SCIENCE
IN
INFORMATION TECHNOLOGY (BSIT)**

Supervisor Signature	In- Charge Examination Signature
Chairman, Department of Information Technology	

Abstract

Mental health issues such as stress, anxiety, and depression have become major concerns among students and young adults, significantly affecting their academic performance, productivity, and overall quality of life. The increasing demand for accessible mental health support highlights the need for innovative digital solutions that can provide timely assistance and promote emotional well-being. This project presents MindMate, a mobile application designed to help users monitor their mental health through mood tracking, self-assessment tools, personalized recommendations, and educational resources. The application was developed using modern software development methodologies with a focus on usability, accessibility, and user engagement. Extensive testing was conducted to evaluate the functionality, performance, and user experience of the system. The results demonstrated that MindMate effectively assists users in tracking their emotional states, increasing self-awareness, and encouraging positive mental health practices through a simple and interactive interface. The findings suggest that MindMate can serve as a reliable and scalable digital mental health companion, contributing to improved emotional well-being while offering significant potential for future integration with artificial intelligence, real-time counseling services, and advanced behavioral analytics.

Acknowledgement

I extend my heartfelt appreciation to my supervisor, Muzammil ur Rahman, for her invaluable guidance, constructive feedback, and unwavering support throughout this project. Her expertise in information Technology and deep learning has been instrumental in shaping my understanding and approach to this complex domain.

I am also grateful to the Department of Information Technology at The Islamia University of Bahawalpur for providing me with the necessary resources, facilities, and academic environment that enabled me to pursue this research effectively.

Special thanks to my family and friends for their constant encouragement, patience, and moral support during the challenging phases of this project. Their belief in my capabilities has been a constant source of motivation throughout my academic journey.

Finally, I acknowledge the broader research community whose published works and open-source contributions in the field of computer vision and deep learning have significantly influenced my project design and implementation.

Kashmala Nawaz
F22BINFT1M01150

TABLE OF CONTENTS

ABSTRACT.....	ERROR! BOOKMARK NOT DEFINED.
---------------	------------------------------

Table of Contents

Supervisor Signature	ii
Chairman, Department of Information Technology	ii
ACKNOWLEDGEMENT	II
1.1 INTRODUCTION	1
1.2 OBJECTIVES	1
1.3 PROBLEM STATEMENT	2
1.4 SCOPE	2
REQUIREMENTS ANALYSIS	3
1.5 LITERATURE REVIEW	3
1.6 USER CLASSES AND CHARACTERISTICS	3
1.6.1 2.2.1 Primary User: University Students (18-26 years)	3
1.6.2 2.2.2 Secondary User: Working Young Adults (26-35 years)	3
1.6.3 2.2.3 Tertiary User: Mental Health Advocates and Counselors	3
1.6.4 2.2.4 Emergency User: Crisis-Affected Individuals	4
1.7 DESIGN AND IMPLEMENTATION CONSTRAINTS	4
1.8 ASSUMPTIONS AND DEPENDENCIES	4
1.9 FUNCTIONAL REQUIREMENTS	4
1.10 USE CASE DIAGRAM	7
1.11 NONFUNCTIONAL REQUIREMENTS	7
1.12 OTHER REQUIREMENTS	8
1.12.1 2.8.1 Data Privacy Requirements	8
1.12.2 2.8.2 Notification Requirements	8
1.12.3 2.8.3 Localization Requirements	8
SYSTEM DESIGN	9
1.13 APPLICATION AND DATA ARCHITECTURE	9
1.13.1 3.1.1 Presentation Layer	9
1.13.2 3.1.2 Domain Layer (Business Logic)	9
1.13.3 3.1.3 Data Layer	9
1.13.4 3.1.4 Firebase Firestore Data Structure	9
1.14 COMPONENT INTERACTIONS AND COLLABORATIONS	9
1.15 SYSTEM ARCHITECTURE	10
1.16 ARCHITECTURE EVALUATION	10
1.16.1 3.4.1 Flutter vs React Native	10
1.16.2 3.4.2 Firebase vs Custom Backend	10
1.17 COMPONENT-EXTERNAL ENTITIES INTERFACE	11
1.18 SCREENSHOTS/PROTOTYPE	11
1.18.1 3.6.1 Workflow	11
1.18.2 3.6.2 Screens	11
1.19 OTHER DESIGN DETAILS	12
1.19.1 3.7.1 UI/UX Design Principles	12
1.19.2 3.7.2 State Management Architecture	12
1.19.3 3.7.3 Streak Algorithm	12
TEST SPECIFICATION AND RESULTS	13

1.20	TEST CASE SPECIFICATION.....	13
1.21	SUMMARY OF TEST RESULTS	15
CONCLUSION AND FUTURE WORK.....		16
1.22	PROJECT SUMMARY	16
1.23	PROBLEMS FACED AND LESSONS LEARNED.....	16
1.23.1	5.2.1 Technical Challenges.....	16
1.23.2	5.2.2 Non-Technical Challenges.....	16
1.24	FUTURE WORK.....	16
1.24.1	5.3.1 AI-Powered Chatbot Integration.....	16
1.24.2	5.3.2 Urdu Language Support.....	17
1.24.3	5.3.3 Wearable Device Integration	17
1.24.4	5.3.4 Professional Therapist Marketplace	17
1.24.5	5.3.5 Community Features	17
REFERENCES.....		18
GLOSSARY		19
DEPLOYMENT/INSTALLATION GUIDE.....		20
1.25	B.1 PREREQUISITES	20
1.26	B.2 STEP 1: CLONE THE REPOSITORY	20
1.27	B.3 STEP 2: FIREBASE SETUP.....	20
1.28	B.4 STEP 3: INSTALL FLUTTER DEPENDENCIES	20
1.29	B.5 STEP 4: RUN THE APPLICATION.....	20
1.30	B.6 STEP 5: BUILD RELEASE APK (ANDROID)	20
1.31	B.7 FIRESTORE SECURITY RULES	21
1.32	B.8 TROUBLESHOOTING	21
LIST OF FIGURES.....		ERROR! BOOKMARK NOT DEFINED.
LIST OF TABLES.....		ERROR! BOOKMARK NOT DEFINED.
CHAPTER 1. INTRODUCTION.....		ERROR! BOOKMARK NOT DEFINED.
1.1	INTRODUCTION	ERROR! BOOKMARK NOT DEFINED.
1.2	OBJECTIVES	ERROR! BOOKMARK NOT DEFINED.
1.3	PROBLEM STATEMENT.....	ERROR! BOOKMARK NOT DEFINED.
1.4	SCOPE.....	ERROR! BOOKMARK NOT DEFINED.
CHAPTER 2. REQUIREMENTS ANALYSIS		ERROR! BOOKMARK NOT DEFINED.
2.1	LITERATURE REVIEW	ERROR! BOOKMARK NOT DEFINED.
2.2	USER CLASSES AND CHARACTERISTICS	ERROR! BOOKMARK NOT DEFINED.
2.3	DESIGN AND IMPLEMENTATION CONSTRAINTS	ERROR! BOOKMARK NOT DEFINED.
2.4	ASSUMPTIONS AND DEPENDENCIES.....	ERROR! BOOKMARK NOT DEFINED.
2.5	FUNCTIONAL REQUIREMENTS	ERROR! BOOKMARK NOT DEFINED.
2.6	USE CASE DIAGRAM	ERROR! BOOKMARK NOT DEFINED.
2.7	NONFUNCTIONAL REQUIREMENTS	ERROR! BOOKMARK NOT DEFINED.
2.8	OTHER REQUIREMENTS	ERROR! BOOKMARK NOT DEFINED.
CHAPTER 3. SYSTEM DESIGN		ERROR! BOOKMARK NOT DEFINED.
3.1	APPLICATION AND DATA ARCHITECTURE	ERROR! BOOKMARK NOT DEFINED.
3.2	COMPONENT INTERACTIONS AND COLLABORATIONS	ERROR! BOOKMARK NOT DEFINED.
3.3	SYSTEM ARCHITECTURE	ERROR! BOOKMARK NOT DEFINED.
3.4	ARCHITECTURE EVALUATION.....	ERROR! BOOKMARK NOT DEFINED.
3.5	COMPONENT-EXTERNAL ENTITIES INTERFACE	ERROR! BOOKMARK NOT DEFINED.

3.6	SCREENSHOTS/PROTOTYPE	ERROR! BOOKMARK NOT DEFINED.
3.7	OTHER DESIGN DETAILS.....	ERROR! BOOKMARK NOT DEFINED.
CHAPTER 4.	TEST SPECIFICATION AND RESULTS	ERROR! BOOKMARK NOT DEFINED.
4.1	TEST CASE SPECIFICATION	ERROR! BOOKMARK NOT DEFINED.
4.2	SUMMARY OF TEST RESULTS.....	ERROR! BOOKMARK NOT DEFINED.
CHAPTER 5.	CONCLUSION AND FUTURE WORK	ERROR! BOOKMARK NOT DEFINED.
5.1	PROJECT SUMMARY	ERROR! BOOKMARK NOT DEFINED.
5.2	PROBLEMS FACED AND LESSONS LEARNED	ERROR! BOOKMARK NOT DEFINED.
5.3	FUTURE WORK	ERROR! BOOKMARK NOT DEFINED.
REFERENCES.		ERROR! BOOKMARK NOT DEFINED.
APPENDIX A	GLOSSARY	ERROR! BOOKMARK NOT DEFINED.
APPENDIX B	DEPLOYMENT/INSTALLATION GUIDE.....	ERROR! BOOKMARK NOT DEFINED.

Revision History

Name	Date	Reason For Changes	Version

1.1 Introduction

Mental health is a fundamental component of overall human wellbeing. According to the World Health Organization (WHO), mental health is defined as a state of well-being in which every individual realizes his or her own potential, can cope with the normal stresses of life, can work productively and fruitfully, and is able to make a contribution to his or her community. Despite this recognition, mental health disorders — including depression, anxiety, stress, and burnout — remain among the leading causes of disability worldwide.

In Pakistan and across South Asia, access to mental health services is severely limited. The ratio of mental health professionals to population is critically low, and cultural stigma further discourages individuals from seeking help. University students, in particular, experience high levels of psychological stress due to academic pressure, financial concerns, social challenges, and uncertainty about the future.

The rapid proliferation of smartphones has created an unprecedented opportunity to deliver mental health support through mobile applications. Digital mental health tools — commonly called mental wellness apps — can provide immediate, stigma-free, and cost-effective support to individuals who might otherwise receive no help at all.

MindMate was conceived as a response to this need. It is a Flutter-based cross-platform mobile application that serves as a personal mental wellness companion. The application provides users with a structured set of tools: daily mood tracking with streak monitoring, guided journaling, evidence-based breathing exercises, a 20-question psychological diagnostic test, personalized insights based on mood history, and an SOS emergency button for crisis situations.

Unlike many wellness apps that are generic or entertainment-oriented, MindMate is grounded in psychological principles and designed specifically for the Pakistani student demographic. It aims to normalize mental health monitoring, empower users with self-awareness, and provide a first line of digital support before professional intervention becomes necessary.

1.2 Objectives

The primary objectives of MindMate are as follows:

- To develop a fully functional, cross-platform mobile application for mental health monitoring and self-care using Flutter.
- To implement a structured daily mood tracking system that records user emotions with timestamps and maintains a streak to encourage consistent engagement.
- To provide guided journaling functionality enabling users to document and reflect on their daily experiences and emotional states.
- To integrate evidence-based relaxation techniques, specifically the 4-7-8 breathing exercise, presented in an interactive and user-friendly format.
- To design and deploy a 20-question Wellness Diagnostic Test based on established psychological frameworks that assesses user personality and stress levels.
- To develop an SOS emergency module providing instant access to mental health crisis resources, helplines, and emergency contacts.
- To build a personalized insights dashboard that visualizes mood trends, streak history, and patterns to promote self-awareness.
- To ensure a secure, privacy-respecting backend using Firebase for authentication and data storage.
- To design an intuitive, accessible, and visually calming user interface that reduces the barrier to daily mental wellness engagement.
- To evaluate the application through functional testing and validate its usability with target users.

1.3 Problem Statement

Mental health disorders among young adults and university students in Pakistan are rising at an alarming rate. Studies suggest that over 34% of university students in Pakistan experience moderate to severe levels of anxiety, while depression rates hover between 20-30% in this demographic. Despite this, the national mental health infrastructure remains critically underdeveloped — with fewer than 500 trained psychiatrists for a population of over 220 million.

The fundamental problem MindMate addresses is the absence of an accessible, affordable, and culturally appropriate digital tool for mental wellness monitoring and first-line self-care. Existing international applications such as Headspace and Calm are often paywalled, language-restricted, or culturally misaligned with Pakistani users. Furthermore, they do not offer emergency response functionality, structured psychological assessments, or offline-capable mood journaling.

MindMate solves this problem by delivering a comprehensive, locally contextualised mental wellness application that users can access freely on both Android and iOS devices. It serves as a daily companion that tracks mood, encourages reflection, teaches relaxation techniques, provides psychological self-assessment, and connects users in crisis with immediate help — all within a single, beautifully designed mobile application.

1.4 Scope

MindMate encompasses the following functional boundaries:

- User account creation and secure authentication via Firebase Authentication.
- Daily mood logging with emoji-based selection, date tracking, and streak management.
- Text-based journaling with secure storage in Firebase Firestore.
- An animated, guided 4-7-8 breathing exercise module with timer.
- A 20-question Wellness Diagnostic Test with automatic scoring and result feedback.
- A mood history screen displaying past entries with dates and emotional states.
- An insights section visualizing mood trends over time.
- An SOS emergency button providing crisis helpline information.
- Settings management including profile, notification preferences, and data privacy.

Out of scope for the current version: AI chatbot integration, real-time therapist chat, prescription management, and integration with wearable devices. These are planned for future iterations.

Requirements Analysis

1.5 Literature Review

The intersection of mobile technology and mental health — commonly referred to as mHealth (mobile health) — has been an active area of research since the mid-2000s. A landmark study by Donker et al. (2013) published in the Journal of Medical Internet Research found that mental health apps can be effective in reducing symptoms of depression and anxiety when designed with evidence-based interventions. This finding laid the theoretical groundwork for the development of consumer-grade mental wellness applications.

Mood tracking, one of MindMate's core features, is grounded in the practice of Ecological Momentary Assessment (EMA) — a research methodology in which users record their emotional states in real time in their natural environment. Research by Myin-Germeys et al. (2018) demonstrated that daily mood monitoring through mobile apps significantly increased users' emotional self-awareness and helped clinicians identify patterns preceding depressive episodes.

The 4-7-8 breathing technique incorporated into MindMate was developed by Dr. Andrew Weil based on Pranayama yoga traditions. Multiple clinical studies have validated its effectiveness in activating the parasympathetic nervous system, reducing cortisol levels, and alleviating acute anxiety within minutes. A 2017 study in Frontiers in Psychology confirmed that slow-paced breathing exercises reduce subjective stress and improve heart rate variability.

In terms of related work, applications such as Headspace (founded 2010) and Calm (founded 2012) pioneered the commercial mental wellness space globally. Woebot (2017) introduced AI-driven cognitive behavioral therapy (CBT) delivery through a chatbot interface. Daylio (2017) popularized emoji-based micro-journaling. However, none of these applications were designed for the Pakistani or South Asian market, lack SOS emergency functionality, and are largely paywalled.

MindMate differentiates itself through its locally relevant design, free accessibility, crisis intervention capability, and integration of multiple evidence-based modalities within a single cross-platform application built specifically for the Flutter ecosystem.

1.6 User Classes and Characteristics

1.6.1 2.2.1 Primary User: University Students (18-26 years)

This is the primary target demographic. These users are digitally native, comfortable with mobile applications, and face elevated mental health risks due to academic pressure. They are likely to engage with mood tracking and journaling features daily and value the streak motivation mechanism. This class requires a visually engaging, low-friction experience.

1.6.2 2.2.2 Secondary User: Working Young Adults (26-35 years)

Young professionals experiencing workplace stress, burnout, or anxiety constitute the secondary user class. These users prioritize quick-access features (breathing exercises, mood logging) and are likely to engage during commutes or work breaks. They appreciate data-driven insights into their emotional patterns.

1.6.3 2.2.3 Tertiary User: Mental Health Advocates and Counselors

This class includes school counselors, mental health volunteers, or community health workers who might recommend MindMate to clients. They interact with the application at a meta-level — understanding its feature set to guide others in using it effectively.

1.6.4 2.2.4 Emergency User: Crisis-Affected Individuals

A small but critical user class comprising individuals who may be in emotional distress or crisis. They interact primarily with the SOS feature. Their needs are simple, urgent, and require zero-friction access to crisis resources. For this class, reliability and speed of the SOS feature are paramount.

1.7 Design and Implementation Constraints

- **Flutter Framework:** The entire application must be built using Flutter (Dart), constraining the UI component library to Flutter's widget ecosystem and third-party Flutter packages.
- **Firebase Backend:** Backend services are limited to the Firebase ecosystem (Authentication, Firestore, Storage), which introduces dependency on Google Cloud services and requires active internet connectivity for most features.
- **Device Compatibility:** The application must support Android 6.0 (API Level 23) and above, and iOS 12.0 and above, limiting the use of APIs introduced in newer OS versions.
- **Storage Limitations:** Firebase Firestore free tier (Spark Plan) limits are a constraint during development — 1 GB storage, 50,000 reads/day, 20,000 writes/day.
- **No Offline Full Support:** Due to Firebase dependency, full offline functionality is limited. Basic mood logging may work offline with sync on reconnection, but real-time features require connectivity.
- **Privacy Compliance:** As the app handles sensitive mental health data, it must comply with Google Play Store and Apple App Store privacy policies, requiring clear data handling disclosure.
- **Development Tools:** VS Code / Android Studio with Flutter SDK 3.x, Dart 3.x, and Git for version control.

1.8 Assumptions and Dependencies

The following assumptions underpin the MindMate requirements:

- Users own a smartphone running Android 6.0+ or iOS 12.0+ with at least 2 GB RAM.
- Users have intermittent or regular internet connectivity for Firebase sync.
- The mental health crisis helpline numbers referenced in the SOS feature are operational and accurate at time of deployment.
- Users have basic literacy in either English or Urdu (current version is English only).
- The 20-question Wellness Diagnostic Test is used for self-awareness purposes only and is not a clinical diagnostic instrument.

Key external dependencies include Firebase SDK for Flutter (firebase_core, firebase_auth, cloud_firestore packages), Flutter pub packages: fl_chart for mood visualization, shared_preferences for local caching, provider or riverpod for state management, Google Play Services on Android devices, and an active Firebase project with Firestore and Authentication enabled.

1.9 Functional Requirements

UC-1: User Registration and Login

Identifier	UC-1
Purpose	Allow new users to create an account and returning users to log in securely
Priority	High
Pre-conditions	App is installed and launched for the first time, or user has logged out
Post-conditions	User is authenticated and redirected to the Home screen
Typical Course of Action	

S# / Actor Action	System Response
1. User opens app and selects Sign Up or Login	System displays registration / login form with email and password fields
2. User enters credentials and taps Submit	System validates input format (email regex, password length >= 8 chars)
3. User confirms action	Firebase Authentication creates/verifies account; user session is initiated
Alternate Course of Action	
Invalid Email	System displays: Please enter a valid email address
Wrong Password	System displays: Incorrect password. Please try again

Table 1: UC-1 — User Registration and Login

UC-2: Mood Tracking

Identifier	UC-2
Purpose	Allow users to log their current mood with an emoji and optional note
Priority	High
Pre-conditions	User is logged in and on the Home screen
Post-conditions	Mood entry is saved to Firestore with timestamp; streak is updated
Typical Course of Action	
S# / Actor Action	System Response
1. User taps Track Today's Mood button on Home screen	System presents mood selection screen with emoji options (Happy, Sad, Anxious, Calm, Angry, Neutral)
2. User selects an emoji representing their mood	System highlights selected emoji and enables optional note field
3. User optionally writes a note and taps Save	System saves entry to Firestore; increments streak counter; navigates back to Home
Alternate Course	
Already Logged Today	System displays today's mood entry and offers Edit option

Table 2: UC-2 — Mood Tracking

UC-3: Guided Breathing Exercise

Identifier	UC-3
Purpose	Guide the user through the 4-7-8 breathing exercise for stress relief
Priority	High
Pre-conditions	User is logged in; Breathe & Relax card is visible on Home screen
Post-conditions	User completes one or more breathing cycles; session duration is recorded
Typical Course of Action	
S# / Actor Action	System Response
1. User taps Breathe & Relax card	System loads the breathing exercise screen with animated visual circle
2. System prompt — Inhale for 4 seconds	Circle expands with animation
3. System prompt — Hold for 7 seconds	Circle pauses

4. System prompt — Exhale for 8 seconds	Circle contracts
5. User taps Repeat or Done	If Done: returns to Home. If Repeat: restarts cycle counter
Alternate — Interruption	User taps back; system saves session length and returns to Home

Table 3: UC-3 — Guided Breathing Exercise

UC-4: Wellness Diagnostic Test

Identifier	UC-4
Purpose	Administer a 20-question psychological assessment and return a wellness score
Priority	Medium
Pre-conditions	User is logged in and navigates to Wellness Diagnostic Tests section
Post-conditions	Test result and score are displayed and stored in user's profile
Typical Course of Action	
S# / Actor Action	System Response
1. User taps Wellness Diagnostic Tests card	System displays instructions and Start button
2. User taps Start	System presents Question 1 of 20 with multiple-choice options and progress bar
3. User answers each question	System advances to next question; records response
4. After Q20	System calculates score; categorizes result (e.g., Low / Moderate / High Stress)
5. Result displayed	Score, category, and recommendations shown on result screen
Alternate — Exit Mid-Test	System prompts: Your progress will be lost. Exit?

Table 4: UC-4 — Wellness Diagnostic Test

UC-5: SOS Emergency Access

Identifier	UC-5
Purpose	Provide instant access to mental health crisis resources for users in distress
Priority	High (Critical Safety Feature)
Pre-conditions	App is open on any screen (SOS button is always visible)
Post-conditions	User is shown crisis helpline numbers and emergency contacts
Typical Course of Action	
S# / Actor Action	System Response
1. User taps the red SOS button (visible on all screens)	System immediately opens SOS screen — no login required for this action
2. System displays crisis resources	Umang helpline (0317-4288665), Rozan Counseling Center, and emergency contacts list
3. User taps a phone number to call directly	System opens device dialer with pre-filled number
Alternate — No Network	Crisis numbers are cached locally; screen is accessible offline

Table 5: UC-5 — SOS Emergency Access

UC-6: View Mood History and Insights

Identifier	UC-6
Purpose	Allow users to review their past mood entries and visualize emotional trends
Priority	Medium
Pre-conditions	User is logged in and has at least one mood entry recorded
Post-conditions	User views chronological mood history and trend charts
Typical Course of Action	
S# / Actor Action	System Response
1. User taps History tab in bottom navigation	System retrieves all mood entries from Firestore for the logged-in user
2. History displayed	List of entries sorted by date (most recent first) with mood emoji and note
3. User taps Insights tab	System renders a line/bar chart of mood scores over the past 7/30 days
Alternate — No Entries	System displays: No mood data yet. Start tracking today!

Table 6: UC-6 — View Mood History and Insights

1.10 Use Case Diagram

The MindMate system boundary encompasses the following actors and use cases:

Primary Actor: Registered User

- Register / Login (UC-1)
- Track Daily Mood (UC-2)
- Use Breathing Exercise (UC-3)
- Take Wellness Diagnostic Test (UC-4)
- Access SOS Emergency Resources (UC-5)
- View History and Insights (UC-6)
- Manage Settings and Profile

Secondary Actor: Firebase (External System)

- Authenticate user credentials
- Store and retrieve Firestore documents
- Manage session tokens

Figure 3: Use Case Diagram — MindMate System (refer to attached diagram)

1.11 Nonfunctional Requirements

Category	Requirement	Target Metric
Performance	App launch time (cold start)	< 3 seconds on mid-range Android
Performance	Mood save response time	< 1 second with active internet
Usability	Task completion rate for new users	> 90% for core features without instruction
Reliability	Firebase uptime dependency	99.9% (per Google Firebase SLA)
Security	User data encryption	All Firestore data encrypted at rest and in transit (TLS)
Security	Authentication	Firebase Auth with email verification

Portability	Supported platforms	Android 6.0+ and iOS 12.0+
Maintainability	Code structure	Clean Architecture with separation of concerns
Scalability	Concurrent users	Firebase scales automatically; app designed for 10,000+ users
Accessibility	Font sizing	Minimum 12pt; scalable with device accessibility settings

Table 7: Non-Functional Requirements Summary

1.12 Other Requirements

1.12.1 2.8.1 Data Privacy Requirements

MindMate stores sensitive personal health data. The application must comply with Google Play Store and Apple App Store data privacy policies. Users must be presented with a clear Privacy Policy at registration explaining what data is collected, how it is stored, and that it is never sold or shared with third parties.

1.12.2 2.8.2 Notification Requirements

The application should support daily push notifications (implemented via Firebase Cloud Messaging) reminding users to track their mood. Notifications must be opt-in and configurable in Settings.

1.12.3 2.8.3 Localization Requirements

The current version supports English only. Future versions should support Urdu to maximize accessibility for Pakistani users.

System Design

1.13 Application and Data Architecture

MindMate employs a layered Clean Architecture pattern, which is the recommended architectural approach for scalable Flutter applications. Clean Architecture divides the codebase into three concentric layers:

1.13.1 3.1.1 Presentation Layer

This is the outermost layer containing all Flutter Widgets (screens and UI components), State Management logic (using Provider or Riverpod), and navigation controllers. Widgets in this layer are 'dumb' — they render data from the state and delegate user actions to the business logic layer. Key screens include: HomeScreen, MoodTrackingScreen, BreathingScreen, DiagnosticTestScreen, HistoryScreen, InsightsScreen, SOSScreen, SettingsScreen.

1.13.2 3.1.2 Domain Layer (Business Logic)

The domain layer contains use case classes that encapsulate the application's business rules. Each use case (e.g., SaveMoodEntryUseCase, GetMoodHistoryUseCase) contains a single execute() method. This layer is entirely independent of Flutter and Firebase — making it highly testable. It also contains entity classes (MoodEntry, UserProfile, DiagnosticResult) defining the core data models.

1.13.3 3.1.3 Data Layer

The data layer implements repository interfaces defined in the domain layer. It communicates with Firebase Firestore (remote data source) and SharedPreferences (local cache). Repository implementations handle data transformation between Firebase documents and domain entities.

1.13.4 3.1.4 Firebase Firestore Data Structure

The Firestore database uses the following collection/document hierarchy:

- users/{userId}/moodEntries/{entryId} — Fields: mood (string), note (string), timestamp (DateTime), streakDay (int)
- users/{userId}/journalEntries/{entryId} — Fields: content (string), timestamp (DateTime), sentiment (string)
- users/{userId}/diagnosticResults/{resultId} — Fields: score (int), category (string), answers (List), timestamp (DateTime)
- users/{userId}/profile — Fields: displayName (string), email (string), joinDate (DateTime), totalStreak (int)

Figure 6: Firebase Realtime Database Structure (refer to attached diagram)

1.14 Component Interactions and Collaborations

The following sequence describes the interaction flow for the core Mood Tracking feature:

- User taps 'Track Today's Mood' on HomeScreen (Presentation Layer)
- HomeScreen notifies MoodProvider (State Management) of navigation intent
- Flutter Navigator pushes MoodTrackingScreen
- User selects emoji and optionally writes note; taps Save
- MoodTrackingScreen calls SaveMoodEntryUseCase.execute(MoodEntry) (Domain Layer)

- SaveMoodEntryUseCase calls MoodRepository.saveMoodEntry(entry) (Data Layer)
- MoodRepository serializes MoodEntry to Map and calls
FirebaseFirestore.instance.collection('moodEntries').add(data)
- Firebase confirms write; repository returns success signal
- Use case updates streak counter via StreakService
- State management updates HomeScreen to reflect new streak count and recent entry

1.15 System Architecture

Component	Technology	Role
Frontend (Mobile App)	Flutter 3.x (Dart)	Cross-platform UI — Android and iOS
State Management	Provider / Riverpod	Reactive state, dependency injection
Authentication	Firebase Authentication	Email/password user auth, session management
Database	Firebase Firestore (NoSQL)	Real-time cloud storage for mood, journal, diagnostic data
Local Storage	SharedPreferences	Offline caching of last mood entry, user preferences
Push Notifications	Firebase Cloud Messaging (FCM)	Daily mood tracking reminders
Charts/Visualization	fl_chart Flutter package	Mood trend line charts and bar graphs
Crisis Data	Hardcoded + Local Cache	SOS helpline numbers (no API dependency)
Version Control	Git + GitHub	Source code management (repo: haseebqureshi92/mindmate)
IDE	VS Code + Flutter Extension	Primary development environment

Table 8: Technology Stack Summary

1.16 Architecture Evaluation

1.16.1 3.4.1 Flutter vs React Native

Criterion	Flutter	React Native
Language	Dart (compiled to native)	JavaScript (bridge to native)
Performance	Near-native (Skia/Impeller renderer)	Slightly lower (JS bridge overhead)
UI Consistency	Pixel-perfect across platforms	Platform-specific components vary
Learning Curve	Moderate (Dart is beginner-friendly)	Lower if JS is known
Community	Rapidly growing (Google-backed)	Mature (Meta-backed)
MindMate Decision	CHOSEN — better performance, consistent UI	—

Table 9: Architecture Evaluation — Flutter vs React Native

1.16.2 3.4.2 Firebase vs Custom Backend

Criterion	Firebase	Custom REST API (Node.js/Django)
Setup Time	Minutes (no server config)	Days/weeks (server, DB, API setup)
Cost	Free tier sufficient for FYP scale	VPS/server costs required
Real-time Sync	Built-in Firestore listeners	Requires WebSockets or polling

Authentication	Pre-built (email, social, phone)	Must build from scratch
Scalability	Auto-scales (Google infrastructure)	Manual scaling required
MindMate Decision	CHOSEN — zero DevOps overhead for student project	—

Table 10: Architecture Evaluation — Firebase vs Custom Backend

1.17 Component-External Entities Interface

- **Firebase Authentication Service:** Communicates over HTTPS using the Firebase SDK. Auth tokens (JWT) are issued on login and refreshed automatically. Interface: `firebaseAuth.signInWithEmailAndPassword(email, password)`.
- **Firebase Firestore:** Communicates over gRPC (internally managed by Firebase SDK). Documents are read using `.get()` and `.snapshots()` stream listeners. Writes use `.set()`, `.add()`, and `.update()` methods.
- **Firebase Cloud Messaging (FCM):** Push notification payloads are sent from Firebase Console or triggered via Cloud Functions. The app registers a device token on startup and handles notification callbacks through `FirebaseMessaging.onMessage`.
- **Device Telephony (SOS Feature):** MindMate uses the `url_launcher` Flutter package to invoke the native phone dialer (`tel:` URI scheme) when a user taps a crisis helpline number. No external API is called — this is a device-level interface.
- **Operating System (Android/iOS):** Flutter interfaces with the OS through platform channels for permissions (notification, camera for future features), local storage, and network access.

1.18 Screenshots/Prototype

1.18.1 3.6.1 Workflow

The MindMate workflow begins with user authentication (login/register). Upon successful login, users land on the Home Screen, which serves as the central hub. From there, users can: (1) log today's mood via the Track Today's Mood button, (2) access the Breathe & Relax module, (3) navigate to the Wellness Diagnostic Test, (4) view their mood history and insights via the bottom navigation bar, (5) access Settings, or (6) activate the SOS emergency button visible on all screens.

1.18.2 3.6.2 Screens

Figure 1 shows the MindMate Home Screen featuring a greeting, motivational quote, mood tracking button, streak card, recent mood entry, and quick-access cards. The SOS button is permanently visible in the bottom-right corner.

Figure 1: MindMate Home Screen — Mood Tracking Interface

Figure 2 shows the Flutter project directory structure for `mindmate_app`, including standard Flutter directories: `android/`, `ios/`, `lib/`, `test/`, and `build/` with key project files.

Figure 2: MindMate App File Structure (Flutter Project)

Figure 7: Mood History and Streak Tracking Screen — displays all past mood entries in a scrollable list, sorted by date descending.

Figure 8: Breathe & Relax — 4-7-8 Exercise Screen — animated circle guiding inhale/hold/exhale cycles.

Figure 9: Wellness Diagnostic Test Screen — 20-question assessment with progress bar and multiple-choice options.

Figure 10: SOS Emergency Screen — crisis helpline numbers with direct dial functionality.

1.19 Other Design Details

1.19.1 3.7.1 UI/UX Design Principles

MindMate's interface was designed around the following principles: (1) Calm Color Palette — predominantly light blue/lavender tones proven to have a soothing psychological effect. (2) Minimal Cognitive Load — each screen has a single primary action to prevent overwhelm. (3) Positive Reinforcement — the streak system and motivational quotes encourage daily return visits. (4) Accessibility — large tap targets (minimum 48x48dp as per Material Design guidelines), readable font sizes, and high contrast ratios.

1.19.2 3.7.2 State Management Architecture

MindMate uses the Provider package for state management. A MoodProvider class extends ChangeNotifier and holds the current mood state, today's entry, and streak count. The HomeScreen listens to MoodProvider via Consumer<MoodProvider> and rebuilds only when relevant state changes, optimizing render performance.

1.19.3 3.7.3 Streak Algorithm

The streak is calculated as follows: on each mood save, the system checks if the user logged a mood on the previous calendar day. If yes, the streak increments by 1. If no entry exists for yesterday, the streak resets to 1 (counting today). The current streak is stored in the user's Firestore profile document and updated on every mood save.

Test Specification and Results

1.20 Test Case Specification

TC-1: User Registration

Identifier	TC-1
Related Requirement(s)	UC-1
Short Description	Verify that a new user can register with valid credentials
Pre-condition(s)	App installed; user has not previously registered with this email
Input Data	Email: testuser@mindmate.com Password: Test@1234
Detailed Steps	1. Launch app 2. Tap Sign Up 3. Enter email and password 4. Tap Register
Expected Result(s)	Account created in Firebase; user redirected to Home screen
Post-condition(s)	User profile document created in Firestore users collection
Actual Result(s)	PASS — Home screen displayed within 2 seconds of registration
Test Case Result	PASSED

Table 11: TC-1 — User Registration

TC-2: Mood Logging

Identifier	TC-2
Related Requirement(s)	UC-2
Short Description	Verify that a user can log their daily mood successfully
Pre-condition(s)	User is logged in; no mood entry exists for today
Input Data	Mood: Happy Note: Feeling great after a good lecture
Detailed Steps	1. Tap Track Today's Mood 2. Select Happy emoji 3. Enter note 4. Tap Save
Expected Result(s)	Mood entry saved; streak increments by 1; Home screen shows new entry under Recent Entry
Post-condition(s)	Firestore moodEntries collection updated with new document
Actual Result(s)	PASS — Entry saved within 800ms; streak updated correctly
Test Case Result	PASSED

Table 12: TC-2 — Mood Logging

TC-3: Breathing Exercise Completion

Identifier	TC-3
Related Requirement(s)	UC-3
Short Description	Verify the 4-7-8 breathing exercise completes one full cycle correctly
Pre-condition(s)	User is on Home screen

Input Data	No input required — animation-driven
Detailed Steps	1. Tap Breathe & Relax card 2. Observe Inhale (4s) 3. Observe Hold (7s) 4. Observe Exhale (8s) 5. Tap Done
Expected Result(s)	Animation completes full 19-second cycle; instructions display in correct sequence; user returned to Home
Post-condition(s)	Session recorded in local SharedPreferences
Actual Result(s)	PASS — Full cycle completed in 19 seconds; all animation phases smooth
Test Case Result	PASSED

Table 13: TC-3 — Breathing Exercise

TC-4: Wellness Diagnostic Test Completion

Identifier	TC-4
Related Requirement(s)	UC-4
Short Description	Verify that the 20-question test is administered and results are shown
Pre-condition(s)	User is logged in
Input Data	Answer all 20 questions with option A (highest stress responses)
Detailed Steps	1. Tap Wellness Diagnostic Tests 2. Tap Start 3. Answer all 20 questions 4. View results
Expected Result(s)	Result screen shows score, category (High Stress), and recommendations
Post-condition(s)	Result stored in Firestore diagnosticResults collection
Actual Result(s)	PASS — All 20 questions displayed; result correctly calculated as High Stress
Test Case Result	PASSED

Table 14: TC-4 — Wellness Diagnostic Test

TC-5: SOS Emergency Button

Identifier	TC-5
Related Requirement(s)	UC-5
Short Description	Verify SOS button is accessible and displays crisis resources
Pre-condition(s)	App is open on any screen
Input Data	None
Detailed Steps	1. Locate red SOS button (bottom-right) 2. Tap SOS 3. View crisis resources screen 4. Tap a phone number
Expected Result(s)	SOS screen opens immediately; helpline numbers are displayed; tapping a number opens device dialer
Post-condition(s)	No data written; device dialer opens with pre-filled number
Actual Result(s)	PASS — SOS screen opened in < 0.5 seconds; dialer opened with correct number
Test Case Result	PASSED

Table 15: TC-5 — SOS Emergency Button

TC-6: Mood History and Insights

Identifier	TC-6
Related Requirement(s)	UC-6
Short Description	Verify mood history displays correctly and insights chart renders
Pre-condition(s)	User has at least 3 mood entries logged on different days
Input Data	No input; retrieval test
Detailed Steps	1. Tap History tab 2. Verify entries listed 3. Tap Insights tab 4. Verify chart displays
Expected Result(s)	History shows all entries sorted by date; Insights shows mood trend chart for past 7 days
Post-condition(s)	No data modified
Actual Result(s)	PASS — 3 entries displayed; chart rendered correctly with correct mood scores
Test Case Result	PASSED

Table 16: TC-6 — Mood History and Insights

1.21 Summary of Test Results

Module Name	Test Cases Run	Number of Defects Found	Number of Defects Corrected	Number of Defects Still to Correct
User Authentication	TC-1	1 (minor: keyboard overlap)	1	0
Mood Tracking	TC-2	0	0	0
Breathing Exercise	TC-3	1 (timer drift on old Android)	1	0
Wellness Diagnostic Test	TC-4	2 (UI scroll issue on small screens)	2	0
SOS Emergency Feature	TC-5	0	0	0
History & Insights	TC-6	1 (chart empty state missing)	1	0
Complete System	TC-1 to TC-6	5 total	5 total	0

Table 17: Summary of All Test Results

Conclusion and Future Work

1.22 Project summary

MindMate successfully addresses a critical gap in the digital mental health landscape for Pakistani university students and young adults. Over the course of this Final Year Project, a fully functional, cross-platform mobile application was designed, developed, tested, and validated using Flutter and Firebase.

The application delivers six core modules: structured mood tracking with streak gamification, guided 4-7-8 breathing exercise, a 20-question Wellness Diagnostic Test, a mood history and insights dashboard, an SOS emergency access feature, and secure user authentication. All six modules passed functional testing with a 100% defect resolution rate.

MindMate demonstrates that a small team of students can develop a meaningful, production-quality mental health application using modern cross-platform frameworks and cloud backend services. The project validated the technical feasibility of deploying mental wellness tools on both Android and iOS from a single Dart codebase, and confirmed that Firebase provides sufficient backend capabilities for the scale of a student mental health application.

Most importantly, MindMate embodies a genuine commitment to improving mental health access for underserved communities. By providing free, accessible, and stigma-free mental wellness tools, it contributes to the broader global effort to democratize mental healthcare through technology.

1.23 Problems faced and lessons learned

1.23.1 5.2.1 Technical Challenges

- **Firebase Firestore Security Rules:** Configuring Firestore security rules to ensure each user can only access their own data proved more complex than anticipated. Lesson: Study Firebase security rules documentation thoroughly before beginning data modeling.
- **Flutter State Management:** Choosing between Provider, Riverpod, Bloc, and GetX was initially confusing. Lesson: For beginner projects, Provider is sufficient; adopt Riverpod for more complex apps as it offers better testability.
- **Animation Synchronization:** Synchronizing the breathing exercise animation with precise 4-7-8 timings required careful use of AnimationController and CurvedAnimation. Lesson: Flutter's animation system is powerful but requires careful planning.
- **Cross-Platform UI Inconsistencies:** Minor rendering differences between Android and iOS required platform-specific adjustments. Lesson: Always test on both platforms throughout development, not just at the end.

1.23.2 5.2.2 Non-Technical Challenges

- **Time Management:** Balancing FYP development with coursework and exams was a persistent challenge. Lesson: Break the project into weekly deliverables with clear milestones from day one.
- **Requirement Creep:** The initial scope was larger (included AI chatbot and Urdu language). These were deferred to future work to maintain focus. Lesson: A well-defined, realistic scope is more valuable than an ambitious but incomplete one.

1.24 Future work

1.24.1 5.3.1 AI-Powered Chatbot Integration

A future version of MindMate will incorporate an AI chatbot powered by a large language model API (such as Claude or GPT-4) to provide empathetic, conversational mental health support. The chatbot will be grounded in Cognitive Behavioral Therapy (CBT) principles and programmed to recognize crisis indicators and escalate to the SOS feature.

1.24.2 5.3.2 Urdu Language Support

To maximize accessibility for Pakistani users, a full Urdu language interface will be developed in the next iteration. This will involve implementing Flutter's localization (intl package) and translating all UI strings and content.

1.24.3 5.3.3 Wearable Device Integration

Integration with wearable devices (Apple Watch, Fitbit, Samsung Galaxy Watch) will allow MindMate to passively collect physiological data (heart rate, sleep patterns) and correlate it with mood data for richer wellness insights.

1.24.4 5.3.4 Professional Therapist Marketplace

A future version will include a directory of licensed mental health professionals in Pakistan, allowing users to book video consultations directly through the app — creating a bridge between self-care tools and professional intervention.

1.24.5 5.3.5 Community Features

Anonymous peer support groups and moderated community forums will be added to reduce isolation and foster connection among users experiencing similar mental health challenges, while maintaining strict privacy and moderation standards.

References

- Donker, T., Petrie, K., Proudfoot, J., Clarke, J., Birch, M. R., & Christensen, H. (2013). Smartphones for smarter delivery of mental health programs: A systematic review. *Journal of Medical Internet Research*, 15(11), e247.
- Kessler, R. C., & Ustun, T. B. (2004). The World Mental Health (WMH) Survey Initiative Version of the World Health Organization (WHO) Composite International Diagnostic Interview (CIDI). *International Journal of Methods in Psychiatric Research*, 13(2), 93-121.
- Myin-Germeys, I., Kasanova, Z., Vaessen, T., Vachon, H., Kirtley, O., Viechtbauer, W., & Reininghaus, U. (2018). Experience sampling methodology in mental health research: New insights and technical developments. *World Psychiatry*, 17(2), 123-132.
- Weil, A. (2015). *Breathing: The master key to self-healing*. Sounds True. Retrieved from <https://www.drweil.com>
- World Health Organization. (2022). *World mental health report: Transforming mental health for all*. WHO Press.
- Google LLC. (2024). Flutter documentation. <https://docs.flutter.dev>
- Google LLC. (2024). Firebase documentation. <https://firebase.google.com/docs>
- Flutter Community. (2024). fl_chart package documentation. https://pub.dev/packages/fl_chart
- Pakistan Association of Mental Health. (2023). *Mental health statistics Pakistan*. <https://pamh.org.pk>

Glossary

Term / Acronym	Definition
Flutter	Google's open-source UI software development toolkit for building natively compiled cross-platform applications from a single Dart codebase
Dart	The programming language used by Flutter; a client-optimized, object-oriented language developed by Google
Firebase	Google's mobile and web application development platform providing backend services including authentication, real-time database, and cloud storage
Firestore	Firebase's scalable NoSQL cloud database for storing and syncing data between users and devices in real-time
FCM	Firebase Cloud Messaging — a cross-platform messaging solution for delivering push notifications
FYP	Final Year Project — a capstone academic project undertaken in the final year of a Bachelor's degree program
mHealth	Mobile Health — the use of mobile devices and applications to support medical and public health practice
EMA	Ecological Momentary Assessment — a research method involving repeated real-time data collection from subjects in their natural environments
CBT	Cognitive Behavioral Therapy — a form of psychological treatment focused on changing unhelpful cognitive distortions and behaviors
4-7-8 Breathing	A breathing technique where the user inhales for 4 seconds, holds for 7 seconds, and exhales for 8 seconds to promote relaxation
SOS	Save Our Souls — in MindMate, refers to the emergency button providing instant access to mental health crisis helplines
API	Application Programming Interface — a set of rules allowing software components to communicate
BSIT	Bachelor of Science in Information Technology — the degree program under which this FYP is submitted
IUB	The Islamia University of Bahawalpur — the academic institution where this project was undertaken
DIT	Department of Information Technology — the department within IUB offering the BSIT degree
UI/UX	User Interface / User Experience — the visual design and interaction design of an application
JWT	JSON Web Token — a compact, URL-safe token format used for securely transmitting authentication information
NoSQL	Not Only SQL — a database category providing storage and retrieval of data modeled differently from tabular relational databases
SDK	Software Development Kit — a collection of software tools and libraries used to develop applications for a specific platform
Streak	In MindMate, a counter tracking the number of consecutive days a user has logged their mood

Table 18: Glossary of Terms

Deployment/Installation Guide

This guide provides step-by-step instructions for developers and evaluators to set up and run the MindMate application from source code.

1.25 B.1 Prerequisites

- Windows 10/11, macOS 12+, or Ubuntu 20.04+ operating system
- Flutter SDK 3.10 or higher installed (<https://docs.flutter.dev/get-started/install>)
- Dart SDK 3.0 or higher (bundled with Flutter SDK)
- Android Studio or VS Code with Flutter and Dart extensions installed
- Android Emulator (API 23+) or physical Android/iOS device
- Git installed (<https://git-scm.com>)
- Active Firebase account (<https://console.firebase.google.com>) — free Spark plan is sufficient

1.26 B.2 Step 1: Clone the Repository

Open a terminal and run: `git clone https://github.com/haseebqureshi92/mindmate.git` then `cd mindmate`

1.27 B.3 Step 2: Firebase Setup

- Go to <https://console.firebase.google.com> and create a new project named 'mindmate'
- Enable Authentication > Email/Password sign-in method
- Enable Firestore Database (Start in test mode for development)
- Download the `google-services.json` file (Android) and place it in: `android/app/google-services.json`
- If building for iOS: download `GoogleService-Info.plist` and place it in: `ios/Runner/GoogleService-Info.plist`

1.28 B.4 Step 3: Install Flutter Dependencies

Run: `flutter pub get`

1.29 B.5 Step 4: Run the Application

Connect a physical device via USB with USB debugging enabled, or start an Android Emulator, then run: `flutter run`

1.30 B.6 Step 5: Build Release APK (Android)

To generate a release APK: `flutter build apk --release`

The output APK will be located at: `build/app/outputs/flutter-apk/app-release.apk`

1.31 B.7 Firestore Security Rules

Before publishing, update Firestore rules in Firebase Console > Firestore > Rules to restrict data access to authenticated users only. Ensure each user can only read and write their own data documents under `users/{userId}/{document=**}`.

1.32 B.8 Troubleshooting

Issue	Solution
'flutter' is not recognized as a command	Add Flutter SDK bin/ folder to system PATH environment variable
google-services.json not found error	Ensure file is placed in android/app/ directory, not project root
Firebase Auth not working	Verify Email/Password sign-in is enabled in Firebase Console
App crashes on launch	Run 'flutter doctor' to diagnose environment issues; check Firebase config
Dependencies not found	Run 'flutter clean' then 'flutter pub get' again
Emulator too slow	Enable Hardware Acceleration (HAXM on Intel) in BIOS and Android Studio